

SEB-XRIF

Scalable, Evidence-Based XR Integration Framework

Technical Specification and Build Plan

Team 3 — PRJT302

Information Technology, Durban University of Technology

Status: Draft for mentor review — v0.1

Scope. This document specifies the software stack, system architecture, model-building protocol, and phased build plan for the analytics and evaluation layers of SEB-XRIF. It is the implementation companion to Section 3 (Research Methodology and Proposed Solution) of the full paper. The data layer is prototyped on the public xAPI Educational Mining Dataset; the VR pilot at DUT is scheduled separately and is out of scope here.

Contents

1	Purpose and Scope	2
1.1	Locked decisions	2
2	System Architecture	3
3	Technology Selection	3
3.1	Data layer	3
3.2	Analytics layer	4
3.3	Service layer	4
3.4	Visualization layer	4
3.5	Evaluation layer	5
3.6	MLOps and infrastructure	5
4	Model-Building Specification	6
4.1	Task definition	6
4.2	Dataset provenance and storage	6
4.3	Preprocessing pipeline	6
4.4	Candidate model matrix	7
4.5	Training and validation protocol	7
4.6	Hyperparameter tuning	7
4.7	Interpretability	7
4.8	Artifacts and metadata	8
4.9	Success criteria	8
5	Service Layer Specification	9
6	Visualization Layer Specification	9
7	Evaluation Layer Specification	9
8	Repository Structure	11
9	Infrastructure and Runtime	11
10	Build Plan	12
10.1	Sequencing rationale	12
11	Risks and Mitigations	12
12	Appendix A: Package Register	14

13 Appendix B: Alternatives Register	15
--	----

1 Purpose and Scope

SEB-XRIF is a reusable way of conducting XR education research, not a single model or dashboard. It has four layers: **data**, **analytics**, **visualization**, and **evaluation**. The artifact delivered by this plan is a reproducible analytics pipeline, a prediction API, a dashboard, and a fixed evaluation protocol. The Random Forest and the dashboard are the first instantiation of the framework; the framework is the recipe others reuse.

The system must satisfy four requirements derived from the systematic literature review and the project SWOT analysis:

1. **Reproducibility.** Any institution can export xAPI data, run the same pipeline with the same seed and configuration, and obtain a comparable result.
2. **Scale.** The pipeline operates on a public dataset larger than the SLR median of 30 participants and is designed for multi-institutional reuse.
3. **Prediction.** The analytics layer predicts an actionable support band from a logged-behaviour model and ranks the behaviours that drive the prediction.
4. **Longitudinal evidence.** The evaluation layer fixes a T0/T1/T2 measurement protocol so retention is measured, closing the SLR's central gap.

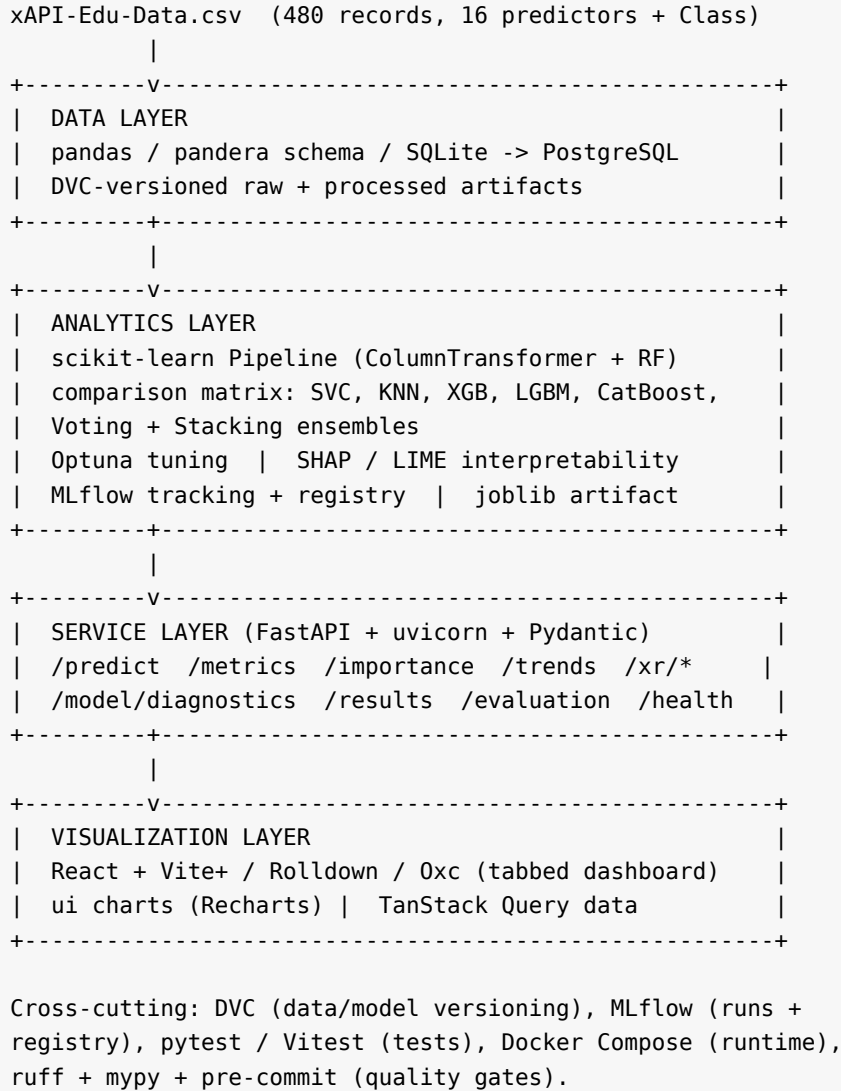
1.1 Locked decisions

The following decisions were agreed with the project owner and are treated as fixed inputs to this specification.

No.	Decision
1	FastAPI as the backend framework (replacing Flask from the draft paper; paper text to be updated).
2	uv -managed fresh Python 3.12 environment (not the system Python 3.14).
3	Full model comparison matrix, including classical baselines, tree ensembles, boosting libraries, and stacked ensembles.
4	MLflow for experiment tracking and model registry.
5	xAPI ingestion through the lrsql Learning Record Store, with a CSV fallback; the ARETE XR pilots are the second source.
6	The ARETE augmented-reality xAPI pilots are the XR transfer demonstration; further secondary datasets (OULAD, UCI) deferred.

2 System Architecture

The architecture maps one-to-one onto the framework's four layers. The analytics and visualization layers are the only ones implemented in this phase; the data layer is exercised through a file loader, and the evaluation layer is implemented as reusable scoring modules.



3 Technology Selection

Each selection below states the chosen tool, the reason, and the candidates that were considered and rejected. Licenses are recorded because the framework is intended for redistribution.

3.1 Data layer

Selected: pandas (DataFrames), pandera (schema validation), SQLAlchemy

1. **psycopg** (persistence), **DVC** (data versioning), **TinCanPython** (tincan)

for xAPI statement modelling, and **SQLite** in development migrating to **PostgreSQL** in deployment.

Why. The dataset is a single 480-row table, so a heavyweight warehouse is unnecessary; **pandas** plus a declarative **pandera** schema gives cheap, testable validation. **SQLAlchemy** abstracts the **SQLite**-to-**PostgreSQL** move so no application code changes. **DVC** makes the raw and processed datasets content-addressed and reproducible, which

is the framework’s core claim. The xAPI standard is represented through `TinCanPython` so the same event schema the pilot will emit can be modelled now without running an LRS.

Alternatives considered: `Polars` (faster, but unnecessary at this size and an extra API for the team to learn); `Great Expectations` (far heavier than `pandera` for a single-table dataset); `DuckDB` (excellent analytics engine, deferred until query volume justifies it); `Git LFS` (weaker pipeline semantics than `DVC`); self-hosted LRS options `lrsql`, `Ralph`, `Learning Locker`, `lxHive`, `educa LRS` (deferred with Decision 5; `lrsql` is the preferred candidate for the pilot because it supports SQLite and PostgreSQL).

3.2 Analytics layer

Selected: `scikit-learn` (pipelines, classical models, ensembles, metrics), `xgboost`, `lightgbm`, `catboost` (boosting), `imbalanced-learn` (class imbalance), `optuna` (hyperparameter search), `shap`, `lime`, `eli5` (interpretability), `joblib` (serialization), `MLflow` (tracking and registry), `matplotlib` + `seaborn` (static diagnostics), `JupyterLab` (exploratory analysis).

Why. `scikit-learn` is the reference implementation for the proposed Random Forest and all classical comparators, and its `Pipeline` guarantees that preprocessing travels with the model, eliminating the most common silent serving bug. The three boosting libraries cover the current state of the art in educational data mining and are all permissive-licensed. `imbalanced-learn` addresses the mild class skew (127 / 211 / 142). `optuna` is materially more sample-efficient than grid search on a small dataset. `shap` provides both global and per-student explanations, which the dashboard uses for the “early warning, not gatekeeping” use case. `joblib` is the recommended serializer for `scikit-learn` artifacts.

Alternatives considered: `pycaret` and `lazypredict` (fast AutoML, but they hide the deliberate model-by-model comparison the paper requires); `autogluon` (powerful, heavyweight, and harder to justify line-by-line); `hyperopt` and `scikit-optimize` (viable tuners, but `optuna` has the better pruning and dashboard); `onnxruntime` + `skl2onnx` (kept as an optional export path, not the primary one); `pickle` (rejected as the primary format because it is less efficient on NumPy arrays and a known deserialization security risk).

3.3 Service layer

Selected: `FastAPI` + `uvicorn[standard]` + `pydantic` / `pydantic-settings`, with `gunicorn` as the process manager in production, `structlog` for structured logs, and `httpx` for the test client.

Why. `FastAPI` provides automatic request validation through `Pydantic`, generates OpenAPI documentation at `/docs` with no extra work, and runs on ASGI with throughput well above a synchronous Flask service — relevant once the pilot points real dashboard traffic at the prediction endpoint. `Pydantic` rejects malformed feature vectors before they reach the model, which is important because education data is often incomplete. This decision replaces Flask in the draft paper and requires a one-word correction in Section 3.

Alternatives considered: `Flask` + `Flask-RESTX` (matches the current paper text, but synchronous and needs validation wired manually); `Django REST`

`Framework` (ORM overhead irrelevant to a stateless inference service); `BentoML` and `KServe` (purpose-built serving, but more infrastructure than a single-model service warrants); `Ray Serve` (for distributed multi-model serving, out of scope).

3.4 Visualization layer

Selected: `React` + `Vite+` (`Vite 8` with the `Rolldown` bundler and `Oxc` tooling) + `TypeScript`, the `ui` design-system chart component (`Recharts`), `TanStack Query` + `axios` for server state, and the `ui` component library for presentation.

Why. `Vite+` is `VoidZero`’s unified toolchain: a single `vp` command covers the dev server, the production build (`Rolldown`, written in Rust), formatting and linting (`Oxfmt` and `Oxlint`, Rust), type checking, tests (`Vitest`), and task running. The Rust components make builds and checks an order of magnitude faster than the previous `esbuild-plus-Rolldown` stack and remove tool drift by keeping configuration in one `vite.config.ts`. Charts use the design system’s chart component (`Recharts`), which is more than adequate for a 480-row dataset. `TanStack Query` centralises fetching, caching, and loading states so the dashboard can be re-skinned without touching data logic.

The design system is delivered as a `shadcn/ui` component library (Base UI + Tailwind v4) under `ui/`, with a `Storybook` story for every component so the UI can be edited and reviewed visually; only the chart wrapper and API contract are fixed.

Alternatives considered: `Chart.js` via `react-chartjs-2` (the original paper choice; superseded by the design system’s `Recharts` chart); `Nivo` (best animation and accessibility, but heavier and `React-Server-Component` incompatible); `Apache`

`ECharts` (the scalability choice for dense dashboards); `visx` (best for bespoke D3 visualisations); `Next.js` (unnecessary SSR for an internal dashboard); `Streamlit` / `Plotly Dash` (fastest to a dashboard, but they cannot host the custom UI kit and would collapse analytics and UI into one non-reusable layer); `MUI` / `Chakra` (rejected because the production UI kit already exists).

3.5 Evaluation layer

Selected: `scikit-learn` metrics, `pingouin` for effect sizes, `scipy` and `statsmodels` for inferential tests, and a small in-repo SUS scorer.

Why. Cohen’s *d* is required by the evaluation protocol, and `pingouin` returns the effect size, confidence interval, and achieved power in one call, which keeps the reporting consistent across adopters. SUS has no maintained library, so a ten-item scorer with the standard odd/even scoring rule is implemented and unit-tested.

Alternatives considered: manual Cohen’s *d* on `scipy` (removes any license concern, kept as a fallback); `statsmodels` alone (lower-level, more code); commercial survey platforms (rejected on cost and reproducibility).

License flag. `pingouin` is GPL-3.0, which is copyleft. This is fine for internal research and the paper, but if SEB-XRIF is later distributed as a product, the effect-size calculation should be swapped to the `scipy` fallback or the GPL obligation accepted deliberately.

3.6 MLOps and infrastructure

Selected: `uv` (Python and dependency management), `ruff` (lint/format), `mypy` (types), `pytest` + `pytest-cov` (Python tests), `Vitest` + `Testing Library` (frontend tests), `pre-commit` (hooks), `Docker` + `Docker Compose` (runtime), `PostgreSQL` (database), `MLflow` (tracking and registry), `DVC` (data and pipeline versioning).

Why. `uv` installs and pins a Python 3.12 toolchain in one command, solving the current environment constraint. `DVC` + `MLflow` together give both data-level and run-level reproducibility, which is what the paper claims. `Docker Compose` defines a single reproducible development and demo environment containing the API, database, and `MLflow` server.

Alternatives considered: `Poetry` and `pip-tools` (both fine; `uv` is faster and also manages the interpreter); `dvclive` (lighter experiment tracking without a server, rejected because Decision 4 chose `MLflow`); `Prometheus` + `Grafana` (deferred observability) — note `Grafana` is AGPL-3.0; `DVC Studio` and `Weights & Biases` (hosted, rejected for data sovereignty and cost); `GitHub Actions` (the CI target once a remote is configured).

4 Model-Building Specification

4.1 Task definition

Supervised multi-class classification. Given 16 predictors describing a learner's demographics, academic context, and logged behaviour, predict a support band derived from the academic `Class` in `{Low, Medium, High}`. The classes are reported as priority-support, monitor, and on-track, and a cost-sensitive threshold can flag priority-support below the argmax.

Property	Value
Target	<code>Class</code> — 3 classes: L (127), M (211), H (142)
Predictors	16: demographics (gender, nationality, place of birth, StageID, GradeID, SectionID), academic context (Topic, Semester, Relation, ParentAnsweringSurvey, ParentschoolSatisfaction, StudentAbsenceDays), behaviour (raisedhands, VisITedResources, AnnouncementsView, Discussion)
Dataset	xAPI Educational Mining Dataset (Kalboard 360), 480 records, CC BY-SA 4.0
Primary metric	Macro F1 and accuracy; expected benchmark 75–83% (Amrieh et al., 2016)

4.2 Dataset provenance and storage

The dataset is downloaded once, hashed, and registered with DVC. A `pandera` schema asserts the 17 columns, their types, and the value domains. The cleaned table is written to SQLite in development and PostgreSQL in deployment through `SQLAlchemy`; no transformation happens in the database, so the pipeline is portable.

4.3 Preprocessing pipeline

All preprocessing is composed inside a single scikit-learn `Pipeline` so that the exact same transformation is applied at training and inference.

```

from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.ensemble import RandomForestClassifier

CATEGORICAL = ["gender", "Nationality", "PlaceofBirth", "StageID", "GradeID",
               "SectionID", "Topic", "Semester", "Relation",
               "ParentAnsweringSurvey", "ParentschoolSatisfaction",
               "StudentAbsenceDays"]
BEHAVIOURAL = ["raisedhands", "VisITedResources",
               "AnnouncementsView", "Discussion"]

tree_pre = ColumnTransformer([
    ("cat", OneHotEncoder(handle_unknown="ignore"), CATEGORICAL),
    ("num", "passthrough", BEHAVIOURAL),
])

pipeline = Pipeline([
    ("pre", tree_pre),
    ("clf", RandomForestClassifier(
        n_estimators=100, class_weight="balanced",
        random_state=42, n_jobs=-1)),
])

```

Rationale. One-hot encoding for categoricals; behavioural counts pass through untouched because tree ensembles are invariant to monotone feature scaling. Standard scaling is applied **only** inside the SVM, KNN, and MLP

branches, where it is required. No imputation stage is needed because the dataset is complete; the schema check proves it rather than assuming it.

4.4 Candidate model matrix

Every model below is trained and evaluated under an identical split, seed, and metric set. This is the full comparison matrix agreed in Decision 3.

Model	Family	Key configuration	Library
Dummy	Baseline	strategy="most_frequent" — floor reference	sklearn
Logistic Regression	Linear	multinomial, max_iter=1000	sklearn
Decision Tree	Tree	criterion="gini", tuned depth	sklearn
Gaussian Naive Bayes	Probabilistic	default	sklearn
Random Forest	Bagged trees	100 trees, class_weight="balanced", seed 42	sklearn
Extra Trees	Bagged trees	n_estimators=100	sklearn
Gradient Boosting	Boosting	tuned learning rate, depth	sklearn
Hist Gradient Boosting	Boosting	fast histogram variant	sklearn
SVC (RBF)	Kernel	C and gamma tuned, probability=True	sklearn
K-Nearest Neighbours	Instance	k tuned, distance weighting	sklearn
XGBoost	Boosting	tuned n_estimators, max_depth, lr	xgboost
LightGBM	Boosting	tuned, leaf-wise	lightgbm
CatBoost	Boosting	tuned, native categoricals	catboost
Voting (soft)	Ensemble	top-3 base learners, soft vote	sklearn
Stacking	Ensemble	base learners + Logistic Regression meta-learner	sklearn
MLP	Neural	hidden_layer_sizes tuned, StandardScaler	sklearn

4.5 Training and validation protocol

1. **Split.** Stratified 80/20 train/test with a fixed seed; the test partition is touched exactly once, at the end, for every model.
2. **Cross-validation.** Stratified 10-fold CV on the training partition for model selection and tuning. Class imbalance is handled inside each fold via `class_weight` (preferred) or `imbalanced-learn` resampling confined to the training folds to prevent leakage.
3. **Metrics.** Accuracy; macro and weighted precision, recall, and F1; confusion matrix; one-vs-rest ROC-AUC and PR-AUC. Macro F1 is the headline metric because the classes are uneven.
4. **Baseline gate.** No model is promoted unless it beats the Dummy classifier and the published 75–83% ensemble benchmark is reported alongside it.
5. **Determinism.** Every estimator receives `random_state=42`; the environment is locked by the `uv` lockfile; each run records its `git` commit and DVC data hash in MLflow.

4.6 Hyperparameter tuning

Optuna performs the search with a TPE sampler and median pruning, using stratified 5-fold CV inside the training partition. Search spaces are small and explicit per model. `GridSearchCV` is retained only for the Random Forest to produce the exact configuration quoted in the paper. Tuning runs are logged to MLflow as nested runs so the chosen configuration is traceable to its evidence.

4.7 Interpretability

1. **Native importance.** Impurity-based and permutation feature importance for the tree models.
2. **SHAP.** Global summary and dependence plots, plus per-student waterfall explanations for the dashboard. Expected leading drivers: `raisedhands`, `VisITedResources`, and `StudentAbsenceDays`.
3. **LIME.** Used as a cross-check on a sample of local explanations.
4. **Constraint.** The dashboard frames predictions as early warning for support, not as gatekeeping; interpretability output carries this framing.

4.8 Artifacts and metadata

Each promoted run writes: the serialized full pipeline (`model.joblib`), a JSON metadata sidecar (git commit, DVC hash, feature order, library versions, metrics, confusion matrix, timestamp), the SHAP explanation payload, and an MLflow run ID. Feature order is stored explicitly and enforced by the API so that column reordering cannot silently corrupt predictions.

4.9 Success criteria

Criterion	Threshold
Predictive performance	Macro F1 and accuracy within or above the 75–83% published benchmark
Reproducibility	<code>dvc repro</code> regenerates identical metrics from a clean checkout
Stability	10-fold CV standard deviation reported; no single-fold collapse
Interpretability	Top drivers are pedagogically plausible and stable across folds
Service parity	API predictions match direct pipeline predictions within numerical tolerance (integration test)

5 Service Layer Specification

The FastAPI service loads the promoted artifact once at startup and exposes the following contract. Input is validated by Pydantic; invalid feature vectors are rejected with HTTP 422 before reaching the model.

Method and path	Purpose	Returns
GET /health	Liveness and model-loaded check	<code>{"status": "ok", "model_version": "..."} </code>
POST /predict	Predict a support band for one learner	Predicted class and support band, per-class probabilities, confidence
POST /predict/batch	Vectorized batch scoring	List of predictions
GET /metrics	Evaluation metrics for the active model	Accuracy, macro/weighted F1, CV mean and std, confusion matrix
GET /importance	Global feature importance and SHAP summary	Ranked feature list with scores
GET /trends	Behavioural trends for dashboard charts	Aggregated time/cohort series from the data layer
GET /model/diagnostics	Out-of-fold model diagnostics	ROC, PR, calibration, ECE, learning curves
GET /results	Comparison matrix and sidecars	Matrix, tuning, explain, evaluation payloads
GET /xr/pilots	Known ARETE XR pilots	Pilot metadata and download status
GET /xr/trends	XR engagement over time	Period buckets, verb counts, top activities
GET /xr/risk	XR early-warning bands	ROC-AUC, average precision, Brier, risk bands
GET /evaluation	Longitudinal evaluation summary	SUS, T0/T1/T2 learning and retention, or an empty state

Operational choices: model loaded once at module scope, not per request; `gunicorn` with `uvicorn` workers for CPU-bound inference; structured request and prediction logging through `structlog`; an integration test that asserts serving parity with the training pipeline; exact dependency pins so the scikit-learn version used for training is the version used for loading.

6 Visualization Layer Specification

The dashboard is a thin client over the API. It has six tabs: Overview (metric cards, support-band distribution, trends, and the longitudinal impact panel), Predict (the support-band form), Data (the paged learner table), Diagnostics (comparison matrix, confusion and calibration, CV spread, importance), Explore (XR engagement trends, XR early-warning risk, the LMS-to-XR feature mapping, correlation, embedding, and partial dependence), and Studio (a chart builder with saved views). A persistent header selector switches the active data source between the LMS seed and the ARETE XR pilots, and the XR panels follow it. Charts use the `ui ChartContainer` wrapper over `Recharts`, so the library can be swapped behind one interface. `TanStack Query` owns all fetching, caching, and retry logic; empty states are explicit, so the dashboard is fully functional before any pilot data exists. The UI kit itself replaces only the presentation layer.

7 Evaluation Layer Specification

The evaluation layer is fixed by the framework: every adopter reports the same measures.

1. **Model metrics.** Accuracy, macro and weighted F1, and 10-fold stratified CV.
2. **Usability.** SUS after the interaction, scored with the standard rule; benchmark 76.6 reported for XR training (Kim et al., 2024).
3. **Learning effect.** Cohen’s d for immediate gain, benchmark 0.936 for immersive practical training (Chang and Hsu, 2023); thresholds 0.2 / 0.5 / 0.8.

4. **Longitudinal protocol.** Three time points — T0 baseline, T1 immediate post, T2 one semester later with no re-teaching. Reported effect sizes: $d_{\text{immediate}} = T1 \text{ vs } T0$, $d_{\text{delayed}} = T2 \text{ vs } T0$, and $T2 \text{ vs } T1$ for decay. T2 is written as a scheduled protocol and future result, not as a fabricated measurement.

8 Repository Structure

```

VR_Education_framework/
  data/
    raw/           # DVC-tracked source CSV (immutable)
    processed/     # DVC-tracked cleaned table
  analytics/
    config.yaml    # seeds, split, model grid, paths
    schema.py      # pandera dataset schema
    preprocess.py  # ColumnTransformer builders
    train.py       # matrix runner + MLflow logging
    tune.py        # Optuna / GridSearchCV
    evaluate.py    # CV, metrics, confusion matrix
    explain.py     # SHAP / LIME payloads
  models/
    model.joblib   # promoted pipeline
    model.meta.json # metadata sidecar
  api/
    main.py        # FastAPI app + startup model load
    schemas.py     # Pydantic request/response models
    routes/        # predict, metrics, trends, xr, evaluation, charts
  web/             # Vite+ / React / TypeScript dashboard
    components/    # ui Card/Badge/Chart panels over the API hooks
    src/api/       # TanStack Query hooks
  eval/
    sus.py         # ten-item SUS scorer
    effect_size.py # pingouin + scipy fallback
    longitudinal/  # T0/T1/T2 schemas and reports
  tests/           # pytest (analytics, api) + Vitest (web)
  dvc.yaml         # reproduce pipeline
  docker-compose.yml # api + postgres + mlflow
  pyproject.toml   # uv-managed dependencies
  uv.lock
  README.md

```

9 Infrastructure and Runtime

Development and demo run through Docker Compose with three services: the FastAPI app, PostgreSQL, and the MLflow tracking server backed by a local artifact store. In development, SQLite and the local MLflow file store are used so no containers are required. Quality gates (ruff, mypy, pytest, Vitest) run through pre-commit. dvc repro is the canonical way to regenerate data, train the promoted model, and reproduce the reported metrics from a clean checkout.

10 Build Plan

The plan is ordered so that each phase produces a demonstrable artifact and a pass/fail exit gate. Phases 0–6 deliver the paper’s technical claims; phases 7–10 harden the system; phase 11 is the separately scheduled pilot.

Phase	Work	Exit gate
0. Environment and scaffold	<code>uv</code> Python 3.12; <code>repo</code> skeleton; <code>pyproject.toml</code> ; <code>ruff</code> , <code>mypy</code> , <code>pre-commit</code> ; <code>dvc</code> <code>init</code> ; <code>git</code> baseline	<code>uv run python -c "import sklearn, fastapi"</code> succeeds; hooks pass
1. Data foundation	Download and hash dataset; <code>pandera</code> schema; loader to SQLite; DVC-track raw and processed	Schema validation passes on all 480 rows; <code>dvc status</code> clean
2. Preprocessing pipeline	<code>ColumnTransformer</code> for categorical and numeric branches; deterministic 80/20 split; leakage-free CV wrapper	Pipeline unit tests pass; split proportions and seed verified
3. Baselines and proposed model	Dummy, Logistic Regression, Decision Tree, Naive Bayes, Random Forest; 10-fold CV	RF beats Dummy and lands in/above the 75–83% band; config recorded
4. Full comparison matrix	SVM, KNN, Extra Trees, Gradient/Hist boosting, XGBoost, LightGBM, CatBoost, Voting, Stacking, MLP; Optuna tuning	Results table complete with CV mean/std; best model promoted or RF retained with justification
5. Interpretability	Native importance; SHAP global and local; LIME cross-check; export payloads	Top drivers plausible and stable; explanation artifact produced
6. Tracking and reproducibility	MLflow server; nested runs for tuning; model registry; <code>dvc.yaml</code> stages	<code>dvc repro</code> reproduces recorded metrics from a clean checkout
7. FastAPI service	App, Pydantic schemas, all endpoints, joblib load-once, structlog; serving-parity test	All endpoints pass tests; parity test within tolerance; <code>/docs</code> renders
8. Dashboard	Vite+ / React / TS scaffold built on the ui components; ui Card/Badge/Chart panels; TanStack Query hooks	All panels populated from live API; design system shared with the ui Storybook
9. Evaluation harness	SUS scorer; Cohen’s d via pingouin with scipy fallback; T0/T1/T2 schemas and report templates	Scorer unit tests pass against known SUS vectors; d matches hand calculation
10. Hardening and documentation	Docker Compose; CI workflow; coverage thresholds; README and runbooks; paper stack corrected to FastAPI	One-command bring-up works; CI green; docs reviewed
11. Pilot integration (future)	Point loader at DUT xAPI data; run T0/T1 through the <code>lrsql</code> LRS; schedule T2	T0/T1 reported; T2 scheduled, not fabricated

10.1 Sequencing rationale

Phases 0–2 remove all environmental and data risk before any modelling. Phase 3 establishes the paper’s core claim on the simplest defensible model. Phase 4 provides the comparative evidence the review demands and the lecturer requested. Phases 5–6 make the result explainable and reproducible, which is the framework’s differentiator. Phases 7–9 deliver the usable system, and phase 10 makes it adoptable. The pilot is deliberately last because it depends on the frozen schema, split, and metrics established in earlier phases.

11 Risks and Mitigations

Risk	Impact	Mitigation
Python 3.14 incompatibility with native ML wheels	Blocks modelling	<code>uv</code> pins a fresh Python 3.12 tool-chain; lockfile freezes it

Risk	Impact	Mitigation
Small dataset limits generalizability	Overstated claims	Report CV spread; compare to published benchmarks; defer secondary datasets per Decision 6 and state the limitation
Class imbalance skews accuracy	Misleading metric	Macro F1 as headline; <code>class_weight</code> and in-fold resampling; report per-class metrics
Feature-order or version mismatch at serving	Silent wrong predictions	Persist feature order in metadata; pin versions; serving-parity integration test
Stack drift from Flask to FastAPI in the paper	Documentation inconsistency	Correct Section 3 and Section 4 text to FastAPI as an explicit phase 10 task
pingouin GPL-3 copyleft	License constraint on redistribution	Scipy fallback implemented and tested; decide before any product release
Scope creep into the VR pilot before the pipeline is frozen	Missed submission	Phase gate: pilot starts only after phase 6 reproduces cleanly

12 Appendix A: Package Register

Python packages are installed from PyPI with `uv`. Pins are exact in `uv.lock`; the versions below indicate the compatibility line.

Package	Pin	License	Role
python	3.12.x	PSF	Interpreter
pandas	2.x	BSD-3	DataFrames and IO
numpy	2.x	BSD-3	Numerics
pandera	0.2x	MIT	Dataset schema validation
pyarrow	1x	Apache-2.0	Parquet and columnar IO
sqlalchemy	2.x	MIT	Database abstraction
alembic	1.x	MIT	Migrations
psycopg[binary]	3.x	LGPL-3	PostgreSQL driver
duckdb	1.x	MIT	Optional analytics engine
dvc	3.x	Apache-2.0	Data and pipeline versioning
tincan	0.0.5	Apache-2.0	xAPI statement modelling
scikit-learn	1.5+	BSD-3	Pipelines, models, metrics
imbalanced-learn	0.12+	MIT	Resampling and imbalance
xgboost	2.x	Apache-2.0	Gradient boosting
lightgbm	4.x	MIT	Gradient boosting
catboost	1.2+	Apache-2.0	Gradient boosting
optuna	4.x	MIT	Hyperparameter search
shap	0.46+	MIT	Global and local explanations
lime	0.2+	BSD-2	Local explanations
eli5	0.13+	MIT	Explanation diagnostics
joblib	1.4+	BSD-3	Model serialization
mlflow	2.x	Apache-2.0	Tracking and model registry
dvclive	3.x	Apache-2.0	Optional lightweight tracking
matplotlib	3.x	PSF-like	Static diagnostics
seaborn	0.13+	BSD-3	Statistical plots
jupyterlab	4.x	BSD-3	Exploration
fastapi	0.11x	MIT	API framework
uvicorn[standard]	0.3x	BSD-3	ASGI server
pydantic	2.x	MIT	Validation and settings
gunicorn	23.x	MIT	Process manager
structlog	24.x	Apache-2.0 / MIT	Structured logging
httpx	0.27+	BSD-3	Test client and calls
pingouin	0.5+	GPL-3	Effect sizes and power
scipy	1.x	BSD-3	Stats fallback
statsmodels	0.14+	BSD-3	Inferential tests
pytest	8.x	MIT	Python tests
pytest-cov	5.x	MIT	Coverage
ruff	0.6+	MIT	Lint and format
mypy	1.x	MIT	Static types
pre-commit	3.x	MIT	Hooks

Package	Pin	License	Role
uv	0.12+	Apache-2.0 / MIT	Env and dependency management

JavaScript packages are installed from npm.

Package	Pin	License	Role
react / react-dom	18.x	MIT	UI runtime
vite-plus / vp	0.3+	MIT	Unified toolchain CLI
@voidzero-dev/vite-plus-core	8.x	MIT	Vite 8 dev server and build
rolldown	1.x	MIT	Rust bundler
oxlint / oxfmt	1.x / 0.6x	MIT	Rust lint and format
typescript	5.x	Apache-2.0	Typed frontend
recharts	3.x	MIT	Charts via the ui chart component
@tanstack/react-query	5.x	MIT	Server state and caching
axios	1.x	MIT	HTTP client
tailwindcss	3.x	MIT	Placeholder styling
shadcn/ui	latest	MIT	Placeholder components
vitest	4.1	MIT	Frontend unit tests
@testing-library/react	16.x	MIT	Component tests

13 Appendix B: Alternatives Register

For traceability, every consideration that was not selected is recorded with the reason.

Area	Candidate	Disposition
DataFrames	Polars	Rejected — unnecessary at 480 rows; extra API
Validation	Great Expectations	Rejected — heavyweight for one table
Analytics DB	DuckDB	Deferred — revisit if query volume grows
Versioning	Git LFS	Rejected — weaker pipeline semantics than DVC
LRS	lrsql, Ralph, Learning Locker, lxHive, educa LRS	Deferred — Decision 5; lrsql preferred for the pilot
AutoML	PyCaret, LazyPredict, AutoGluon	Rejected — obscure the deliberate comparison required by the paper
Tuning	Hyperopt, scikit-optimize, GridSearchCV	Mostly rejected — Optuna more sample-efficient; GridSearchCV retained for RF only
Serialization	pickle, onnxruntime + skl2onnx	pickle rejected as primary (security, efficiency); ONNX kept as optional export
Backend	Flask, Django REST, BentoML, KServe, Ray Serve	Rejected — FastAPI chosen per Decision 1
Charts	Chart.js (original paper), Nivo, ECharts, visx	Recharts selected via the ui design-system chart; others rejected or superseded
App framework	Next.js	Rejected — SSR unnecessary for an internal dashboard
Dashboard platforms	Streamlit, Plotly Dash	Rejected — cannot host the custom UI kit; would collapse layers
Tracking	dvclive, DVC Studio, Weights & Biases	dvclive kept optional; hosted tools rejected for cost and data sovereignty
Observability	Prometheus + Grafana	Deferred — note Grafana is AGPL-3.0
Effect size	scipy manual, statsmodels	Kept as fallback; pingouin selected despite GPL-3 for completeness

Area	Candidate	Disposition
Frontend tooling	Vite 6 + Rollup + esbuild, Vitest 2	Replaced by Vite+ (Vite 8 / Rolldown / Oxc) for speed and a single toolchain
CI	GitHub Actions	Configured: CI runs on every push and pull request